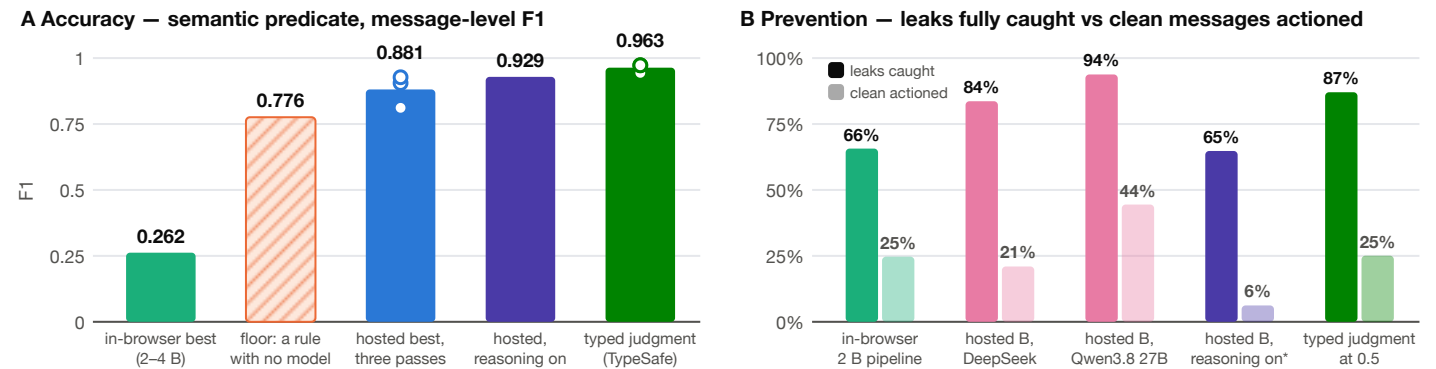


Can small language models in the browser prevent data leakage in LLM prompts — and does a larger, or a different kind of, model change the answer?

AI-DLPP study · four in-browser 2–4 B models on WebGPU in real Chrome, the identical method on five hosted open-weight models (27–120 B where published) over three passes, and a typed-judgment model (TypeSafe Jev) over three passes · 189-message corpus, blind-annotated gold, a model-free floor beside every number · every figure regenerated from artifacts · draft v3, 17 Sep 2026

- Q1. Can small language models, running in real time directly in the browser, identify and prevent data leakage?
- Q2. Does the result differ if we use larger models instead of in-browser ones?
- Q3. Does it differ again if the model returns typed judgments instead of text?



Same corpus, policy and gold. Hatched = a rule with no model. B = policy-in-context. Typed judgment = TypeSafe at 0.5. \*Reasoning-on counts its 59 unanswered rows as misses.

Accuracy and prevention, side by side. (A) F1 at the policy's semantic predicate, message level — best in-browser arm, the model-free floor, the best hosted arm over three passes (dots), and the hosted model with reasoning allowed to finish. (B) Share of leak-bearing messages fully caught (dark) against share of clean messages wrongly actioned (light), entity level. Time and cost are in the table below.

|             | Q1 · SMALL MODELS, IN THE BROWSER (2–4 B)                                                                                                                                                                                | Q2 · LARGER MODELS, HOSTED (27–120 B)                                                                                                                                                                                                                      | Q3 · TYPED JUDGMENT, HOSTED (JEV)                                                                                                                                                                     |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accuracy    | 0.262 vs floor 0.776 — semantic predicate F1; every in-browser model is below a trivial capitalisation rule, because it fires on most messages. Entity spans: F1 0.44–0.49, of which the regex tier alone supplies 0.47. | 0.881 vs floor 0.776 — DeepSeek, mean of three passes; the other four never clear the floor (27 B sits on it, 120 B at 0.30–0.47). With reasoning, a sixth model reaches recall 1.000 (F1 0.929). Entity spans 0.675 model-only, 2.0× the in-browser best. | 0.963 vs floor 0.776 — mean of three passes at the untuned default, and 0 false alarms in 160 clean messages on every pass. Perfect ranking; 0.971 when the threshold is picked on held-out messages. |
| Prevention  | 62–67% of leak-bearing messages fully caught, 25% of clean messages wrongly actioned — the same 20 of 81, produced by the regex tier whichever model sits behind it.                                                     | 84% caught at 21% over-blocked (DeepSeek); 94% at 44% (Qwen3.8 27B). Larger models find more; how eagerly they fire is model-specific.                                                                                                                     | 87% caught at 25% flagged; turn the threshold up and it is 78% at 12%, beating the regex tier on both at once.                                                                                        |
| Feasibility | Runs in real Chrome on WebGPU. Only the 2 B model answers every message inside the 5 s budget; 3–4 B models exceed it on 3–70 of 189; policy-in-context prompting never reaches a first token on three of four.          | Not shippable — prompts would leave the machine; the answer points to a local server running the same weights. 0 malformed responses in 5,580 calls; rate limits cost 1.8% of rows.                                                                        | Not shippable either — also hosted. 567 calls, 0 errors, 0 rate limits, 189 of 189 answered every pass. One request per message.                                                                      |
| Cost        | \$0 per message. Hardware: 2.2–3.4 GB VRAM, 1.7–3.6 s engine load; the compiled prompt is 408–447 tokens against ~1,570 for policy-in-context.                                                                           | \$0.04–0.31 per 1,000 messages by model (compiled judge \$0.09, policy-in-context \$0.21, 2.5×); reasoning \$0.23–1.44. The whole study, 6,502 calls: \$1.22. Local-server capital cost not measured.                                                      | \$0.12 per 1,000 messages. Three passes over the whole corpus cost 7 cents.                                                                                                                           |
| Time        | 1.0 s median, 2.6 s p95 per message on the 2 B model; 2.6–4.5 s median on 3–4 B with p95 at the budget; policy-in-context 3.3 s on the 116 messages it answers, 4.4 s over all 189.                                      | 0.36–1.2 s median per call for the compiled judge, 0.75–4.5 s for policy-in-context — the same order as the browser's best, not faster. The judge answers in 5–7 tokens: ~0.1 s of decode on a 60 tok/s local server. Reasoning: 3.7–10 s per message.     | 0.28 s median per message, 0.33–0.40 s p95 — the fastest arm in the study.                                                                                                                            |

Conclusion

Q1 — partly. A 2 B model checks every message inside a second at no cost, and the shippable pipeline stops two leak-bearing messages in three while interrupting one clean prompt in four. That protection is the regex tier's: no in-browser model beats a trivial rule at the policy's judgment call. Structured identifiers yes, a client relationship in prose no.

Q2 — yes, on accuracy. One hosted model of five, DeepSeek, clears the floor in every pass and catches five leaks in six at lower over-blocking; the two with published sizes never clear it. Latency matches the browser's best, at cents per thousand messages. Hosted inference is forbidden by the cloud boundary, so this describes what a local server would deliver.

Q3 — yes, and it is the biggest change. Dropping generated text, and letting code propose spans while a judgment model only keeps or rejects them, clears the floor by a wider margin than any language model here, never flags a clean message, and moves 0.029 between passes where the language models moved up to 0.344. Fastest arm in the study, and hosted, so it inherits Q2's boundary.

Verdict: ship the in-browser tier for what it does well — real-time identifier catching at 1 s and \$0 — and put the semantic judgment on a server you control. The design that transfers is the shape rather than the vendor: code proposes the candidates, a small judgment model keeps or rejects them, and the threshold stays yours to set.

Bounds: one policy, one predicate, 19 gold positives (one item moves recall by 0.053); the judgment arm's span recall is capped by the candidates code proposes; the corpus generator gives every positive a capitalised organisation name, which flatters both the floor and the separation; the reasoning result is one pass with 31% of rows lost to rate limiting. Seven defects in the study's own instrumentation are stated in the paper (§7). Repository [orkancelikhisar/ai-dlpp](#); full records in [docs/research/](#).